

iam-jit + NanoClaw V2

Protocol-aware audit + scoped credentials for your AI agents

2026-05-23

iam-jit + NanoClaw V2

Protocol-aware audit + scoped credentials for your AI agents.

iam-jit works alongside your existing credential vault — OneCLI, Bitwarden, HashiCorp Vault, or your own infrastructure. We add the protocol-aware audit + scoped role layer for AWS / K8s / SQL / HTTPS that complements whatever credential layer you’ve chosen.

This walkthrough shows the five-minute setup against a real NanoClaw V2 deployment plus the operator value: action-level audit, per-task scoped roles with TTL, cross-protocol session correlation, and dynamic deny rules — all self-hosted, all Apache-2.0, no SaaS dependency.

Section 1 — What iam-jit adds to your NanoClaw deployment

iam-jit ships four bouncers — one per protocol your agent reaches over the network:

Bouncer	Protocol	What it audits
ibounce	AWS API (SigV4)	s3:GetObject, ec2:RunInstances, etc.
kbounce	Kubernetes API	verb + resource + namespace (get pods/...)
dbounce	SQL wire (PG/MySQL)	statement + table + WHERE-clause shape
gbounce	HTTPS	CONNECT host:port (TLS passthrough)

Each bouncer is a standard system proxy: agents reach AWS via `AWS_ENDPOINT_URL`, K8s via `KUBECONFIG`, HTTPS via `HTTPS_PROXY`. NanoClaw’s container honors these by default — no NanoClaw modification needed.

What you get on top of “the call ran”:

- **Action-level audit.** Not “agent talked to AWS” but “agent called `s3:GetObject` on `arn:aws:s3:::backups-prod/2026-05-22.tar.gz` from session `f3a1...`”. SigV4 / K8s API verbs / SQL ASTs decoded at the protocol layer, emitted as OCSF v1.1.0 events.
- **Per-task scoped IAM roles with TTL.** iam-jit *creates* short-lived roles fresh for each task (read-only by default, scoped to the resources the task actually needs). It never mutates existing roles.
- **Cross-protocol session correlation.** One `agent.session_id` threaded through `X-Agent-Session-Id` (HTTP) and `application_name=iam-jit-agent:NAME:SESSIONID` (SQL). One

query (iam-jit audit query --filter agent.session_id=...) returns events from all four protocols.

- **Dynamic deny rules.** Operator says “Claude, make sure this doesn’t touch prod for 3h” → MCP tool fires → all four bouncers honor the deny + any role iam-jit issues during the window embeds an explicit Deny statement (defense-in-depth, post-#324f).
- **OCSF v1.1.0 audit emission.** Drop the JSONL into Splunk, Security Lake, Panther — schema-stable, already-mapped.
- **Self-host + free + open source (Apache-2.0).** Bouncers run on your machine. No phone-home. No external SaaS dependency.

Three operator modes, measured hit-rates across an adversarial corpus: **38.5% in discovery default** (observe + audit only) / **69.2% with one dynamic-deny rule** / **84.6% with an audit-pinned profile**. Pick the mode that fits your adoption phase. (Source: tests/dogfood/role-effectiveness-grades-post-pivot.md, 13-scenario corpus, four bouncers, graded MEANINGFUL / PARTIAL / THEATER / NEGATIVE-VALUE.)

Section 2 — Five-minute setup

Prerequisites

- Docker (Docker Desktop or colima) — for NanoClaw + LocalStack containers
- Python 3.11+ — for ibounce
- Go 1.21+ — for kbounce, dbounce, gbounce
- AWS access OR LocalStack (this walkthrough uses LocalStack; the recipe is identical against real AWS)

Install the iam-jit Bouncer suite

Pre-release install from source. PyPI / Homebrew releases land at v1.0; for now, install from source.

```
# Demo workspace — generic path so others can follow along
mkdir -p /tmp/iam-jit-demo && cd /tmp/iam-jit-demo

# ibounce — AWS API gating (Python)
git clone https://github.com/trsreagan3/iam-jit.git
cd iam-jit
python3 -m venv .venv && source .venv/bin/activate
pip install -e .
which ibounce          # → /tmp/iam-jit-demo/iam-jit/.venv/bin/ibounce

# kbounce — Kubernetes API gating (Go)
cd /tmp/iam-jit-demo
git clone https://github.com/trsreagan3/kbouncer.git
cd kbouncer && make build
ls bin/kbounce          # → built binary
```

```
# dbounce - SQL wire gating (Go)
cd /tmp/iam-jit-demo
git clone https://github.com/trsreagan3/dbounce.git
cd dbounce && make build
```

```
# gbounce - HTTPS proxy (Go)
cd /tmp/iam-jit-demo
git clone https://github.com/trsreagan3/gbounce.git
cd gbounce && make build
```

Bring up LocalStack (the demo AWS environment)

Demo uses LocalStack for AWS service isolation; recipe is identical against real AWS. Just point ibounce's --upstream at <https://your-region.amazonaws.com> instead of LocalStack.

```
docker run --rm -d --name iam-jit-demo-localstack \
  -p 4566:4566 \
  -e SERVICES=s3,iam,sts \
  localstack/localstack:latest
```

```
# Seed a demo bucket so the agent has something to audit
```

```
docker exec iam-jit-demo-localstack \
  awslocal s3 mb s3://backups-demo
```

```
docker exec iam-jit-demo-localstack \
  awslocal s3 cp /etc/hostname s3://backups-demo/2026-05-22.tar.gz
```

Start the four bouncers

```
mkdir -p ~/.iam-jit/audit
```

```
# ibounce on 8767 - AWS API
```

```
ibounce run \
  --port 8767 \
  --upstream http://host.docker.internal:4566 \
  --profile safe-default \
  --audit-log-path ~/.iam-jit/audit/ibounce.jsonl &
```

```
# kbounce on 8766 - K8s API
```

```
/tmp/iam-jit-demo/kbouncer/bin/kbounce run \
  --port 8766 \
  --profile safe-default \
  --kubeconfig ~/.kube/config \
  --audit-log-path ~/.iam-jit/audit/kbounce.jsonl &
```

```
# dbounce on 5433 (wire) / 8768 (mgmt) - SQL
```

```
/tmp/iam-jit-demo/dbounce/bin/dbounce run \
  --port 5433 --mgmt-port 8768 \
```

```

--upstream postgresql://demo@host.docker.internal:5432/demo \
--profile safe-default \
--audit-log-path ~/.iam-jit/audit/dbounce.jsonl &

# gbounce on 8080 (data) / 8769 (mgmt) – HTTPS
/tmp/iam-jit-demo/gbounce/bin/gbounce run \
--port 8080 --mgmt-port 8769 --allow-connect \
--audit-log-path ~/.iam-jit/audit/gbounce.jsonl &

```

Launch NanoClaw V2 with bouncer routing

Follow NanoClaw’s quick-start for the latest install command. The relevant container env-var addition for iam-jit composition:

```
SID="$(uuidgen)"
```

```

docker run --rm \
-e AWS_ENDPOINT_URL=http://host.docker.internal:8767 \
-e KUBECONFIG=/path/to/kbounce-routed-config \
-e DATABASE_URL=postgresql://demo@host.docker.internal:5433/demo \
-e HTTPS_PROXY=http://host.docker.internal:8080 \
-e HTTP_PROXY=http://host.docker.internal:8080 \
-e NO_PROXY=localhost,127.0.0.1 \
-e X_AGENT_NAME=nanoclave \
-e X_AGENT_SESSION_ID="$SID" \
nanoclave:latest

```

Set up once. Bouncers run as standard system proxies (AWS_ENDPOINT_URL, KUBECONFIG, HTTPS_PROXY). NanoClaw’s container honors these by default — no NanoClaw modification needed.

X-Agent-Name + X-Agent-Session-Id are set per the agent-identity-in-audit convention so every event across every bouncer carries attribution.

Verify

```

$ curl -s http://127.0.0.1:8767/healthz      # ibounce
{"ok":true,"version":"1.0.0","mode":"cooperative"}

$ curl -s http://127.0.0.1:8766/healthz      # kbounce
{"ok":true,"version":"1.0.0"}

$ curl -s http://127.0.0.1:8768/healthz      # dbounce mgmt
{"ok":true,"version":"1.0.0"}

$ curl -s http://127.0.0.1:8769/healthz      # gbounce mgmt
{"ok":true,"version":"1.0.0"}

```

All four bouncers respond on their listed ports. Open <http://127.0.0.1:8767/>, [:8766/](http://127.0.0.1:8766/), [:8768/](http://127.0.0.1:8768/), [:8769/](http://127.0.0.1:8769/) in a browser for the live audit-stream UIs (refresh every 2s, color-coded by verdict).

Section 3 — Run a task: see iam-jit working

A realistic incident-response task that touches multiple protocols:

“Audit our backup S3 bucket and spot-check the api-staging deployment.”

Inside NanoClaw, the agent runs (roughly):

```
# Step 1 – list the backup bucket
aws --endpoint-url=$AWS_ENDPOINT_URL s3 ls s3://backups-demo/

# Step 2 – fetch the most recent backup metadata
aws --endpoint-url=$AWS_ENDPOINT_URL s3 cp \
  s3://backups-demo/2026-05-22.tar.gz - | head -c 1024 > /tmp/preview

# Step 3 – describe the K8s deployment
kubectl get deployment api-staging -n staging -o yaml

# Step 4 – check the backup-status badge (HTTPS to internal dashboard)
curl https://internal-status.example.com/backups/latest
```

Tail the audit stream in another terminal:

```
$ iam-jit audit stream --filter agent.session_id=$SID
```

ts	bouncer	verdict	actor	operation
07:54:11Z	ibounce	ALLOW	nanoclaw/f3a1...	s3:ListBucket
arn:aws:s3:::backups-demo				
07:54:12Z	ibounce	ALLOW	nanoclaw/f3a1...	s3:GetObject
arn:aws:s3:::backups-demo/2026-05-22.tar.gz				
07:54:14Z	kbounce	ALLOW	nanoclaw/f3a1...	get deployments/api-staging (ns: staging)
07:54:16Z	gbounce	ALLOW	nanoclaw/f3a1...	CONNECT internal-status.example.com:443

One OCSF event in full (the s3:GetObject row):

```
{
  "metadata": {
    "version": "1.1.0",
    "product": {"name": "ibounce", "vendor_name": "iam-jit", "version": "1.0.0"}
  },
  "time": 1716450852000,
  "class_uid": 6003,
  "class_name": "API Activity",
  "activity_name": "GetObject",
  "type_uid": 600302,
  "type_name": "API Activity: Read",
  "severity": "Informational",
  "status_id": 1,
```

```

"status": "Success",
"actor": {
  "user": {"name": "demo-operator", "uid": "demo-operator"},
  "session": {"uid": "req-7e0c1f"}
},
"api": {
  "operation": "s3:GetObject",
  "service": {"name": "s3"},
  "request": {"uid": "42"}
},
"resources": [{
  "name": "2026-05-22.tar.gz",
  "uid": "arn:aws:s3::backups-demo/2026-05-22.tar.gz",
  "type": "s3 resource"
}],
"src_endpoint": {"ip": "127.0.0.1", "port": 51234},
"dst_endpoint": {"hostname": "s3.us-east-1.amazonaws.com"},
"unmapped": {
  "iam_jit": {
    "mode": "cooperative",
    "profile": "safe-default",
    "verdict": "ALLOW",
    "decision_id": 42,
    "enforced": false,
    "agent": {
      "name": "nanoclaw",
      "session_id": "f3a1a02e-1d4f-4f88-b56a-7c2d9d3e1c01",
      "detected_from": "X-Agent-Session-Id"
    },
    "ext": {"aws_region": "us-east-1"}
  }
}
}

```

That single event tells your SIEM:

- **Who:** agent nanoclaw, session f3a1..., operator demo-operator
- **What:** s3:GetObject on arn:aws:s3::backups-demo/2026-05-22.tar.gz
- **Verdict:** ALLOW under profile safe-default
- **Why visible:** SigV4 was decoded at the proxy; the action + ARN are pulled from the wire, not inferred from logs after the fact

Cross-protocol correlation, one filter:

```
$ iam-jit audit query --filter agent.session_id=$SID
```

```

ibounce → 2 events (1 ListBucket, 1 GetObject)
kbounce → 1 event  (1 get deployments)
gbounce → 1 event  (1 CONNECT)
dbounce → 0 events

```

total → 4 events across 3 protocols, 1 session

Section 4 — Add a dynamic deny mid-task

Mid-task, the operator notices the agent’s task description was ambiguous about which environment it should touch. They tell Claude:

“Make sure this doesn’t touch prod for the next 3h.”

Claude (or any MCP-capable agent) calls iam-jit’s bounce_deny_add MCP tool, which fans out the deny across all four bouncers:

```
$ iam-jit deny add \  
  --action 's3:*' \  
  --resource 'arn:aws:s3::*-prod*' \  
  --ttl 3h \  
  --reason "operator: no prod writes during incident response"
```

```
ok dyn-deny-rule-id: d8c1...  
  propagated: ibounce ✓ kbounce ✓ dbounce ✓ gbounce ✓  
  expires:    2026-05-23T10:54:32Z
```

Now the agent attempts a s3:DeleteObject on a prod-data-bucket (perhaps mis-typing the bucket name during cleanup):

```
$ aws --endpoint-url=$AWS_ENDPOINT_URL s3 rm s3://backups-prod-data/2026-05-22.  
tar.gz
```

An error occurred (AccessDenied) when calling the DeleteObject operation:
Access denied by iam-jit dynamic deny rule d8c1...

The audit event surfaces both the deny and its provenance:

```
{  
  "activity_name": "DeleteObject",  
  "status_id": 4,  
  "status": "Denied",  
  "api": {"operation": "s3:DeleteObject"},  
  "resources": [{"uid": "arn:aws:s3::backups-prod-data/2026-05-22.tar.gz"}],  
  "unmapped": {  
    "iam_jit": {  
      "verdict": "DENY",  
      "deny_source": "dynamic",  
      "dynamic_deny_rule_id": "d8c1...",  
      "deny_reason": "operator: no prod writes during incident response",  
      "agent": {  
        "name": "nanoclaw",  
        "session_id": "f3a1a02e-1d4f-4f88-b56a-7c2d9d3e1c01"  
      }  
    }  
  }  
}
```

```

    }
  }
}

```

Defense-in-depth (post-#324f): the next role iam-jit issues during the 3h window embeds the explicit Deny statement in the role's inline policy, so even if the agent finds an out-of-band code path to AWS, the issued credentials carry the deny *as IAM policy* — not just as a request-time check.

```

{
  "Effect": "Deny",
  "Action": "s3:*",
  "Resource": "arn:aws:s3:::*-prod*",
  "Condition": {
    "DateLessThan": {"aws:CurrentTime": "2026-05-23T10:54:32Z"}
  }
}

```

Two enforcement layers, one operator action.

Section 5 — Pin the work as a reusable profile

The incident is wrapped up. The shape of the task — list backups bucket + fetch one object + describe one deployment + hit one HTTPS endpoint — is something the operator wants to be able to run as a scoped credential next time, instead of relying on default discovery scoping.

```

$ iam-jit profile generate-from-audit \
  --session $SID \
  --out /tmp/iam-jit-demo/incident-foo-recovery.profile.yaml

```

Analyzing 4 events across 3 protocols...

LLM-generated profile + rationale written to:

```

/tmp/iam-jit-demo/incident-foo-recovery.profile.yaml

```

Summary:

- 1 s3:ListBucket on backups-demo
- 1 s3:GetObject under backups-demo/*
- 1 K8s get deployments in namespace=staging
- 1 HTTPS CONNECT to internal-status.example.com:443
- 0 SQL queries

Operator reviews the generated YAML, narrows two wildcards, and saves it:

```

$ iam-jit-bouncer profile save \
  --from /tmp/iam-jit-demo/incident-foo-recovery.profile.yaml \
  --as incident-foo-recovery

```

ok profile installed: incident-foo-recovery

activate with: ibounce profile activate incident-foo-recovery

Next time this incident pattern fires, the operator (or the agent, via MCP) activates the saved profile and the next session gets scoped credentials automatically — no manual scope-narrowing, no broader discovery role lying around.

The **84.6% hit-rate** in the role-effectiveness corpus is conditioned on this audit-pinned profile workflow (post-#326). Default discovery remains 38.5%; one dynamic-deny rule lifts to 69.2%; an audit-pinned profile lands at 84.6%. Operators choose the mode that fits their adoption phase.

Section 6 — What you get vs not (honest scope statement)

iam-jit lives at the network-protocol layer. We're explicit about what that does and doesn't cover.

With iam-jit

- **Protocol-decoded audit** for AWS (SigV4), K8s (verb-resource-namespace), SQL (statement + table + WHERE shape), and HTTPS (CONNECT host:port).
- **Scoped TTL roles** issued fresh per task; never narrowed-from-existing.
- **Cross-protocol session correlation** via a single `agent.session_id` query across all four bouncers.
- **Dynamic deny rules** fan out across the suite + embed into any role iam-jit issues during the deny window.
- **OCSF v1.1.0-shaped events** for direct ingest into Splunk, Security Lake, Panther, etc.
- **Self-host, Apache-2.0, no phone-home.** Bouncers run on your machine or in your cluster; no SaaS dependency.

Not with iam-jit (other tools handle these — that's fine)

- **Credential vault for messaging apps** (Slack, WhatsApp, Telegram, Gmail tokens). OneCLI Agent Vault and similar gateways are excellent for this; iam-jit doesn't store those secrets.
- **Filesystem-level credential blocking.** Agent harness dashboards (openclaw-mission-control, qwibitai/nanoclaw-dashboard, etc.) cover internal state + filesystem audit.
- **HITL approval cards in Slack/Teams.** iam-jit has a request + approve flow for IAM grants; if you need rich Slack approval UX, combine with the harness tool that already does that.
- **Multi-tenant SaaS dashboard.** iam-jit is self-host first. There is no hosted multi-tenant offering by design.

Run alongside your credential vault of choice. iam-jit lives at the protocol layer they don't gate. We add audit + scoped roles for AWS / K8s / SQL / HTTPS to complement whatever credential layer NanoClaw is already plumbing.

Section 7 — Where to go next

Repositories (all Apache-2.0)

- **iam-jit (ibounce + core):** <https://github.com/trsreagan3/iam-jit>
- **kbounce (K8s):** <https://github.com/trsreagan3/kbouncer>

- **dbounce (SQL):** <https://github.com/trsreagan3/dbounce>
- **gbounce (HTTPS):** <https://github.com/trsreagan3/gbounce>

Documentation

- **Canonical NanoClaw + OpenClaw integration recipe:** docs/INTEGRATION-OPENCLAW-NANOCLAW.md in the iam-jit repo. Covers Path A (Chain) / Path B (Replace) / Path C (Parallel) with verified test commands.
- **Dynamic deny rules:** docs/DYNAMIC-DENY-RULES.md
- **Cross-bouncer audit queries:** docs/IAM-JIT-AUDIT-QUERY.md
- **Profile generation from audit:** docs/PROFILE-GENERATION.md
- **Agent attribution conventions:** docs/AGENT-ATTRIBUTION.md

Works with other agent harnesses too

The same `AWS_ENDPOINT_URL` / `KUBECONFIG` / `HTTPS_PROXY` env-var pattern is honored by Claude Code, Cursor, Devin, OpenClaw, Open Interpreter, and any agent or CLI that respects standard system proxies. The NanoClaw V2 recipe in this PDF is one concrete install context; the same shape works wherever your agent runs.

License

All four bouncers ship under Apache-2.0. Self-host has zero billing dependency on iam-jit-the-company; commercial features (Pro-tier LLM-augmented risk scoring, Enterprise self-host license + support) are opt-in and additive.

Contact

Issues + feature requests: <https://github.com/trsreagan3/iam-jit/issues>. Partnership inquiries: open an issue tagged partnership and we'll route from there.